

Федеральное государственное автономное образовательное учреждение высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Отчёт по лабораторной работе №6

Вычислительная математика

Вариант 8

Выполнил:
Караганов Павел

Группа: Р3210

Проверил:
Рыбаков Степан Дмитриевич

г. Санкт-Петербург,

2026

1 Описание алгоритмов решения (Вариант 8)

В данной работе для решения задачи Коши $y' = f(x, y), y(x_0) = y_0$ используются следующие численные методы:

1.1 Усовершенствованный метод Эйлера (метод с пересчетом)

Это метод 2-го порядка точности. Алгоритм на каждом шаге h :

1. Прогноз: $\tilde{y}_{i+1} = y_i + hf(x_i, y_i)$
2. Коррекция: $y_{i+1} = y_i + \frac{h}{2}(f(x_i, y_i) + f(x_{i+1}, \tilde{y}_{i+1}))$

1.2 Метод Рунге-Кутты 4-го порядка

Является классическим одношаговым методом. Вычисляются четыре коэффициента: $k_1 = hf(x_i, y_i)$ $k_2 = hf(x_i + \frac{h}{2}, y_i + \frac{k_1}{2})$ $k_3 = hf(x_i + \frac{h}{2}, y_i + \frac{k_2}{2})$ $k_4 = hf(x_i + h, y_i + k_3)$ Итоговое значение: $y_{i+1} = y_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$

1.3 Многошаговый метод Адамса (Предиктор-корректор)

Используется 4-шаговая схема. Первые три точки (y_1, y_2, y_3) вычисляются методом Рунге-Кутты 4-го порядка. Далее используется пара формул:

- Предиктор (Экстраполяционная формула Адамса-Башфорта):

$$y_{i+1}^{(p)} = y_i + \frac{h}{24}(55f_i - 59f_{i-1} + 37f_{i-2} - 9f_{i-3})$$

- Корректор (Интерполяционная формула Адамса-Мултона):

$$y_{i+1} = y_i + \frac{h}{24}\left(9f\left(x_{i+1}, y_{i+1}^{(p)}\right) + 19f_i - 5f_{i-1} + f_{i-2}\right)$$

2 Рабочие формулы оценки погрешности

1. Для одношаговых методов (Эйлер, РК4) используется правило Рунге:

$$R = \frac{|y_h - y_{\frac{h}{2}}|}{2^{p-1}}, \text{ где } p - \text{порядок точности метода.}$$

2. Для многошагового метода Адамса точность оценивается по сравнению с точным решением (если оно известно):

$$\varepsilon = \max_{0 \leq i \leq n} |y_i^{\text{точн}} - y_i^{\text{прибл}}|$$

3 Программная реализация

Ниже представлен код на языке Python, реализующий выбранные методы.

```
import numpy as np
import matplotlib.pyplot as plt
import warnings
import os

warnings.filterwarnings("ignore", category=RuntimeWarning)

MIN_COORD = -1000.0
MAX_COORD = 1000.0
MIN_H = 0.00001
MAX_H = 100.0
MIN_EPS = 1e-7
```

```
MAX_EPS = 1.0
```

```
class ODESolver:
```

```
    def __init__(self, f):  
        self.f = f
```

```
    def improved_euler(self, x0, y0, xn, h):  
        steps = int(round(abs(xn - x0) / h))  
        if steps == 0: return np.array([x0]), np.array([y0])  
        x = np.linspace(x0, xn, steps + 1)  
        y = np.zeros(steps + 1)  
        y[0] = y0  
        for i in range(steps):  
            k1 = self.f(x[i], y[i])  
            y_pred = y[i] + h * k1  
            y[i+1] = y[i] + (h/2) * (k1 + self.f(x[i+1], y_pred))  
        return x, y
```

```
    def runge_kutta_4(self, x0, y0, xn, h):  
        steps = int(round(abs(xn - x0) / h))  
        if steps == 0: return np.array([x0]), np.array([y0])  
        x = np.linspace(x0, xn, steps + 1)  
        y = np.zeros(steps + 1)  
        y[0] = y0  
        for i in range(steps):  
            k1 = h * self.f(x[i], y[i])  
            k2 = h * self.f(x[i] + h/2, y[i] + k1/2)  
            k3 = h * self.f(x[i] + h/2, y[i] + k2/2)  
            k4 = h * self.f(x[i] + h, y[i] + k3)  
            y[i+1] = y[i] + (k1 + 2*k2 + 2*k3 + k4) / 6  
        return x, y
```

```
    def adams_method(self, x0, y0, xn, h):  
        steps = int(round(abs(xn - x0) / h))  
        if steps < 4:  
            return self.runge_kutta_4(x0, y0, xn, h)
```

```
        x = np.linspace(x0, xn, steps + 1)  
        y = np.zeros(steps + 1)
```

```
        _, y_rk = self.runge_kutta_4(x0, y0, x0 + 3*h, h)  
        y[:4] = y_rk
```

```
        f_vals = [self.f(x[i], y[i]) for i in range(4)]
```

```
        for i in range(3, steps):  
            y_pred = y[i] + (h/24) * (55*f_vals[3] - 59*f_vals[2] + 37*f_vals[1] -  
9*f_vals[0])
```

```
            x_next = x[i+1]  
            f_next_pred = self.f(x_next, y_pred)
```

```
            y[i+1] = y[i] + (h/24) * (9*f_next_pred + 19*f_vals[3] - 5*f_vals[2] +  
f_vals[1])
```

```
            f_vals.pop(0)
```

```

        f_vals.append(self.f(x[i+1], y[i+1]))

    return x, y

def get_runge_error(self, method_name, x0, y0, xn, h, p):
    method = getattr(self, method_name)
    _, y1 = method(x0, y0, xn, h)
    _, y2 = method(x0, y0, xn, h/2)

    if np.isinf(y1[-1]) or np.isnan(y1[-1]) or np.isinf(y2[-1]) or
np.isnan(y2[-1]):
        return float('inf')

    return abs(y1[-1] - y2[-1]) / (2**p - 1)

def safe_input_float(prompt, min_val, max_val):
    while True:
        raw_val = input(prompt).strip()
        if raw_val.lower() == 'q':
            return None

        raw_val = raw_val.replace(',', '.')

        try:
            val = float(raw_val)
            if min_val <= val <= max_val:
                return val
            else:
                print(f"[-] Ошибка: Введите число в диапазоне от {min_val} до
{max_val}")
        except ValueError:
            print("[-] Ошибка: Некорректный формат числа. Попробуйте еще раз.")

def safe_input_int(prompt, valid_choices):
    while True:
        raw_val = input(prompt).strip()
        if raw_val.lower() == 'q':
            return None

        try:
            val = int(raw_val)
            if val in valid_choices:
                return val
            else:
                print(f"[-] Ошибка: Выберите один из вариантов: {valid_choices}")
        except ValueError:
            print("[-] Ошибка: Введите целое число.")

def get_exact_solution(choice, x0, y0):
    if choice == 1:
        # y' = y + (1+x)y^2
        if y0 == 0:
            return lambda x: np.zeros_like(x)
        c = (1/y0 + x0) * np.exp(x0)
        return lambda x: 1 / (c * np.exp(-x) - x)
    elif choice == 2:

```

```

        #  $y' = x^2 - 2y$ 
        c = (y0 - 0.5*x0**2 + 0.5*x0 - 0.25) * np.exp(2*x0)
        return lambda x: 0.5*x**2 - 0.5*x + 0.25 + c * np.exp(-2*x)
    elif choice == 3:
        #  $y' = y + x$ 
        c = (y0 + x0 + 1) * np.exp(-x0)
        return lambda x: -x - 1 + c * np.exp(x)

def main_loop():
    print("ЛАБОРАТОРНАЯ РАБОТА: ЧИСЛЕННОЕ РЕШЕНИЕ ОДУ")

    equations = {
        1: lambda x, y: y + (1+x)*y**2,
        2: lambda x, y: x**2 - 2*y,
        3: lambda x, y: y + x
    }

    while True:
        print("\nВыберите уравнение:")
        print("1:  $y' = y + (1+x)y^2$ ")
        print("2:  $y' = x^2 - 2y$ ")
        print("3:  $y' = y + x$ ")

        choice = safe_input_int("Уравнение (1/2/3): ", [1, 2, 3])
        if choice is None: break

        x0 = safe_input_float(f"x0 (от {MIN_COORD} до {MAX_COORD}): ", MIN_COORD,
MAX_COORD)
        if x0 is None: continue

        y0 = safe_input_float(f"y0 (от {MIN_COORD} до {MAX_COORD}): ", MIN_COORD,
MAX_COORD)
        if y0 is None: continue

        xn = safe_input_float(f"xn (от {x0 + MIN_H} до {MAX_COORD}): ", x0 + MIN_H,
MAX_COORD)
        if xn is None: continue

        h = safe_input_float(f"Шаг h (от {MIN_H} до {MAX_H}): ", MIN_H, min(MAX_H, xn -
x0))
        if h is None: continue

        eps = safe_input_float(f"Точность eps (от {MIN_EPS} до {MAX_EPS}): ", MIN_EPS,
MAX_EPS)
        if eps is None: continue

        f = equations[choice]
        exact_func = get_exact_solution(choice, x0, y0)
        solver = ODESolver(f)

        methods = [
            ("improved_euler", "Усовершенствованный Эйлер", 2),
            ("runge_kutta_4", "Рунге-Кутта 4 пор.", 4),
            ("adams_method", "Адамс (Предиктор-корректор)", 4)
        ]

```

```

plt.figure(figsize=(10, 6))

try:
    x_fine = np.linspace(x0, xn, 500)
    y_fine = exact_func(x_fine)
    plt.plot(x_fine, y_fine, 'k--', label="Точное решение", alpha=0.5)
except Exception:
    pass

print("\n" + "-"*40)
print("РЕЗУЛЬТАТЫ:")
for m_id, name, p in methods:
    try:
        x_res, y_res = getattr(solver, m_id)(x0, y0, xn, h)

        if np.any(np.isinf(y_res)) or np.any(np.isnan(y_res)):
            print(f"{name}: [!] Решение расходится (достигнута бесконечность)")
            continue

        if m_id != "adams_method":
            err_runge = solver.get_runge_error(m_id, x0, y0, xn, h, p)
            err_runge_str = f"{err_runge:.6g}"
        else:
            err_runge_str = "N/A"

        y_exact = exact_func(x_res)
        abs_err = np.max(np.abs(y_res - y_exact))

        print(f"{name}:")
        print(f"    Макс. отклонение от точного: {abs_err:.6g}")
        print(f"    Оценка погрешности по Рунге: {err_runge_str}")

        plt.plot(x_res, y_res, 'o-', label=name, markersize=4)
    except Exception as e:
        print(f"{name}: [!] Ошибка вычисления ({e})")

print("-" * 40)

plt.title(f"Решение ОДУ {choice} при x0={x0}, y0={y0}, h={h}")
plt.legend()
plt.grid(True)
script_dir = os.path.dirname(os.path.abspath(__file__))
img_dir = os.path.join(script_dir, "img")
os.makedirs(img_dir, exist_ok=True)
out_path = os.path.join(img_dir, "interpolation.png")
plt.savefig(out_path, dpi=150, bbox_inches="tight")
plt.close()

if __name__ == "__main__":
    main_loop()

```

4 Результаты работы и выводы

В ходе тестирования программы на уравнении $y' = y + x$ с начальным условием $y(0) = 1$ были получены следующие результаты:

- **Метод Рунге-Кутты 4-го порядка** показал наивысшую точность при заданном шаге h .

- **Метод Адамса** требует разгона другим методом, но вычислительно менее затратен на длинных интервалах, так как требует меньше вычислений функции $f(x, y)$ на шаг.
- **Усовершенствованный метод Эйлера** заметно уступает в точности методам 4-го порядка, что подтверждается правилом Рунге.

Вывод: Численные методы позволяют эффективно решать ОДУ, однако выбор метода зависит от требуемой точности и вычислительной сложности задачи. Метод Рунге-Кутты остается «золотым стандартом» для большинства инженерных задач.